

Testing React Apps

Unit Tests: A Complete Guide to Writing Reliable Tests

Contents

1	What & Why Unit Tests?	2
1.1	Understanding Unit Tests	2
1.2	Why Write Unit Tests?	2
1.3	Real-World Example	2
1.4	The Testing Mindset	3
2	Understanding Different Kinds Of Tests	4
2.1	Unit Tests	4
2.2	Integration Tests	4
2.3	End-to-End (E2E) Tests	4
2.4	The Testing Pyramid	5
2.5	Comparing Test Types	5
3	What To Test & How To Test	6
3.1	What Should You Test?	6
3.2	What NOT to Test	6
3.3	Testing Philosophy: Behavior vs Implementation	6
3.4	The AAA Pattern	6
4	Understanding the Technical Setup & Involved Tools	8
4.1	Common Testing Tools	8
4.1.1	Test Runners	8
4.1.2	Testing Libraries	8
4.1.3	Assertion Libraries	8
4.2	Setting Up Jest	9
4.2.1	Installation	9
4.2.2	Jest Configuration	9
4.2.3	Setup File	9
4.3	Understanding the Test Environment	9
5	Running a First Test	10
5.1	A Simple Function Test	10
5.2	Your First Component Test	10
5.3	Running Tests	11

6	Grouping Tests Together With Test Suites	12
6.1	Using describe() for Organization	12
6.2	Before and After Hooks	13
6.3	Skipping and Only Running Tests	13
7	Testing User Interaction & State	15
7.1	Finding Elements	15
7.2	Testing User Input	15
7.3	Testing Button Clicks	16
7.4	Testing Form Submissions	16
7.5	Testing State Changes	17
8	Testing Connected Components	18
8.1	Testing with Props	18
8.2	Testing with Context	18
8.3	Testing with Redux or State Manager	19
8.4	Testing with React Router	19
9	Testing Asynchronous Code	20
9.1	Testing Promises	20
9.2	Testing API Calls	20
9.3	Testing Loading States	21
9.4	Testing Error States	21
9.5	Testing useEffect	21
10	Working With Mocks	23
10.1	Understanding Mocks	23
10.2	Creating Mock Functions	23
10.3	Mock Return Values	23
10.4	Mocking Modules	24
10.5	Mocking API Calls	24
10.6	Mocking Child Components	24
10.7	Mocking Window Objects	25
10.8	Mock Assertions	25
11	Common Jest Matchers	27
11.1	Basic Matchers	27
11.2	Number Matchers	27
11.3	String Matchers	27
11.4	Array Matchers	28
11.5	DOM Matchers	28
12	Best Practices & Summary	29
12.1	Testing Best Practices	29
12.2	Common Mistakes	29
12.3	Testing Checklist	30
12.4	Test Coverage Goals	30
12.5	Quick Reference	30

13 Conclusion**31**

Chapter 1

What & Why Unit Tests?

Unit tests are automated tests that verify small pieces of your code work correctly. Instead of manually clicking buttons and checking if things work, you write code that automatically tests your code.

1.1 Understanding Unit Tests

A unit test focuses on testing one small piece of functionality in isolation. For example:

- Does a function return the correct value?
- Does a component render with the right content?
- Does clicking a button trigger the right function?

1.2 Why Write Unit Tests?

- **Catch Bugs Early:** Tests find problems before users do
- **Confidence to Change Code:** Refactor without fear of breaking things
- **Documentation:** Tests show how code is supposed to work
- **Save Time:** Automated tests are faster than manual testing
- **Prevent Regressions:** Make sure old bugs don't come back
- **Easier Debugging:** Failed tests point directly to the problem
- **Professional Quality:** Companies expect tests in real projects

1.3 Real-World Example

Imagine a shopping cart component. Without tests, you'd need to:

1. Add item manually
2. Remove item manually

3. Check quantity manually
4. Verify price calculation manually
5. Do this every time you change code

With tests, you write code once that checks all these automatically every time.

1.4 The Testing Mindset

Good tests answer: “What should this code do, and does it actually do it?”
Not: “Did I write this code correctly?”

Chapter 2

Understanding Different Kinds Of Tests

2.1 Unit Tests

Test one small piece of code in isolation.

- **Focus:** Single function or component
- **Speed:** Very fast (milliseconds)
- **Dependencies:** Isolated from other parts
- **Example:** Testing if `add(2, 3)` returns 5

2.2 Integration Tests

Test multiple pieces working together.

- **Focus:** Multiple components or functions
- **Speed:** Slower than unit tests
- **Dependencies:** Tests real interactions
- **Example:** Testing a form that validates, calls API, and updates UI

2.3 End-to-End (E2E) Tests

Test the entire application from user perspective.

- **Focus:** Complete user workflows
- **Speed:** Slowest (seconds or minutes)
- **Dependencies:** Real browser, real server
- **Example:** User logs in, buys product, receives email confirmation

2.4 The Testing Pyramid

The Testing Pyramid:

- Bottom: Many Unit Tests (70%)
- Middle: Some Integration Tests (20%)
- Top: Few E2E Tests (10%)

Why? Unit tests are fast and cheap. Focus there first.

2.5 Comparing Test Types

Type	Speed	Complexity	Coverage
Unit	Very Fast	Simple	Specific
Integration	Moderate	Moderate	Broad
E2E	Slow	Complex	Complete

Chapter 3

What To Test & How To Test

3.1 What Should You Test?

- **Critical Features:** Login, payment, data validation
- **Complex Logic:** Calculations, algorithms, state management
- **Edge Cases:** Empty inputs, null values, very large numbers
- **User Interactions:** Clicking, typing, submitting forms
- **Error Handling:** What happens when something goes wrong?

3.2 What NOT to Test

- **Third-Party Libraries:** React, jQuery, etc. are already tested
- **Minor Styling:** Don't test CSS colors or exact spacing
- **Implementation Details:** Test behavior, not how it works internally
- **Everything:** 100% coverage is unnecessary (aim for 80%)

3.3 Testing Philosophy: Behavior vs Implementation

Test What Matters:

BAD: Test that the component calls `setState`

GOOD: Test that the UI updates when the user clicks

3.4 The AAA Pattern

Structure your tests with three parts:

1. **Arrange:** Set up test data and components
2. **Act:** Perform the action being tested

3. **Assert:** Check if the result is correct

Example:

```
// Arrange: Create test data
const numbers = [1, 2, 3];

// Act: Call the function
const result = sum(numbers);

// Assert: Check the result
expect(result).toBe(6);
```

Chapter 4

Understanding the Technical Setup & Involved Tools

4.1 Common Testing Tools

4.1.1 Test Runners

A test runner finds and executes your tests.

- **Jest:** Most popular for React, built-in to Create React App
- **Vitest:** Modern alternative, very fast
- **Mocha:** Flexible test runner, needs more setup

4.1.2 Testing Libraries

Libraries to test React components and DOM interaction.

- **React Testing Library:** Recommended, tests like users
- **Enzyme:** Older approach, tests implementation details
- **@testing-library/react:** Official testing library for React

4.1.3 Assertion Libraries

Check if test results match expectations.

- **Jest Assertions:** Built into Jest, very complete
- **Chai:** Popular assertion library
- **Expect:** Common syntax across tools

4.2 Setting Up Jest

4.2.1 Installation

Jest comes built-in with Create React App:

```
npx create-react-app my-app
# Jest is already included!
```

Or install manually:

```
npm install --save-dev jest @testing-library/react @testing-library/jest-dom
```

4.2.2 Jest Configuration

Create a `jest.config.js` file:

```
module.exports = {
  testEnvironment: 'jsdom',
  setupFilesAfterEnv: ['<rootDir>/src/setupTests.js'],
  moduleNameMapper: {
    '\\\\.(css|less)$': 'identity-obj-proxy',
  },
};
```

4.2.3 Setup File

Create `src/setupTests.js`:

```
import '@testing-library/jest-dom';

// This file runs before tests, setting up the environment
```

4.3 Understanding the Test Environment

- **jsdom:** Simulates a browser in Node.js
- **React Test Utilities:** Provides helpers to render and interact with components
- **DOM Queries:** Methods to find elements by text, role, label, etc.

Chapter 5

Running a First Test

5.1 A Simple Function Test

Let's test a utility function:

```
// utils.js
export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}
```

Now write tests:

```
// utils.test.js
import { add, multiply } from './utils';

test('add should return the sum', () => {
  expect(add(2, 3)).toBe(5);
});

test('multiply should return the product', () => {
  expect(multiply(2, 3)).toBe(6);
});
```

Run the tests:

```
npm test
```

5.2 Your First Component Test

```
// Button.js
export function Button({ label, onClick }) {
  return <button onClick={onClick}>{label}</button>;
}
```

Test the component:

```
// Button.test.js
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Button } from './Button';

test('renders button with label', () => {
  render(<Button label="Click me" onClick={() => {}} />);
  const button = screen.getByText('Click me');
  expect(button).toBeInTheDocument();
});

test('calls onClick when clicked', async () => {
  const handleClick = jest.fn();
  render(<Button label="Click me" onClick={handleClick} />);

  const button = screen.getByText('Click me');
  await userEvent.click(button);

  expect(handleClick).toHaveBeenCalledTimes(1);
});
```

5.3 Running Tests

```
# Run all tests
npm test

# Run tests in watch mode (reruns on file change)
npm test -- --watch

# Run specific test file
npm test Button.test.js

# Run with coverage
npm test -- --coverage
```

Chapter 6

Grouping Tests Together With Test Suites

6.1 Using describe() for Organization

Group related tests together:

```
// Calculator.test.js
import { add, subtract, multiply, divide } from './Calculator';

describe('Calculator', () => {
  describe('Addition', () => {
    test('adds positive numbers', () => {
      expect(add(2, 3)).toBe(5);
    });

    test('adds negative numbers', () => {
      expect(add(-2, -3)).toBe(-5);
    });

    test('adds zero', () => {
      expect(add(5, 0)).toBe(5);
    });
  });

  describe('Multiplication', () => {
    test('multiplies positive numbers', () => {
      expect(multiply(2, 3)).toBe(6);
    });

    test('multiplies by zero', () => {
      expect(multiply(5, 0)).toBe(0);
    });
  });

  describe('Division', () => {
    test('divides numbers', () => {
      expect(divide(10, 2)).toBe(5);
    });
  });
});
```

```
    test('throws error on division by zero', () => {
      expect(() => divide(10, 0)).toThrow();
    });
  });
});
```

6.2 Before and After Hooks

Set up and clean up for tests:

```
describe('User Management', () => {
  let database;

  // Runs before each test
  beforeEach(() => {
    database = new MockDatabase();
    database.connect();
  });

  // Runs after each test
  afterEach(() => {
    database.disconnect();
  });

  // Runs once before all tests
  beforeAll(() => {
    console.log('Starting user tests');
  });

  // Runs once after all tests
  afterAll(() => {
    console.log('Finished user tests');
  });

  test('creates a new user', () => {
    const user = database.createUser('Alice');
    expect(user.name).toBe('Alice');
  });

  test('deletes a user', () => {
    database.createUser('Bob');
    database.deleteUser('Bob');
    expect(database.getUser('Bob')).toBeNull();
  });
});
```

6.3 Skipping and Only Running Tests

```
// Skip a test
test.skip('this test will be skipped', () => {
  expect(true).toBe(false);
});
```



```
});  
  
// Run only this test  
test.only('only this test runs', () => {  
  expect(true).toBe(true);  
});  
  
// Skip entire suite  
describe.skip('These tests are skipped', () => {  
  test('this wont run', () => {});  
});
```

Chapter 7

Testing User Interaction & State

7.1 Finding Elements

React Testing Library provides queries to find elements:

```
import { render, screen } from '@testing-library/react';
import { Form } from './Form';

test('finding elements', () => {
  render(<Form />);

  // Find by text
  const button = screen.getByText('Submit');

  // Find by placeholder
  const input = screen.getByPlaceholderText('Enter name');

  // Find by label
  const email = screen.getByLabelText('Email');

  // Find by role (best practice)
  const checkbox = screen.getByRole('checkbox');
});
```

7.2 Testing User Input

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { TextInput } from './TextInput';

test('typing in input updates value', async () => {
  const handleChange = jest.fn();
  render(<TextInput onChange={handleChange} />);

  const input = screen.getByRole('textbox');

  await userEvent.type(input, 'Hello');
```

```
    expect(input.value).toBe('Hello');
    expect(handleChange).toHaveBeenCalled();
  });
```

7.3 Testing Button Clicks

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Counter } from './Counter';

test('clicking button increments counter', async () => {
  render(<Counter />);

  const incrementBtn = screen.getByRole('button', { name: /increment/i });
  const display = screen.getByText('0');

  expect(display).toHaveTextContent('0');

  await userEvent.click(incrementBtn);

  expect(screen.getByText('1')).toBeInTheDocument();
});
```

7.4 Testing Form Submissions

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { LoginForm } from './LoginForm';

test('submitting form calls handler', async () => {
  const handleSubmit = jest.fn();
  render(<LoginForm onSubmit={handleSubmit} />);

  const emailInput = screen.getByLabelText('Email');
  const passwordInput = screen.getByLabelText('Password');
  const submitBtn = screen.getByRole('button', { name: /login/i });

  await userEvent.type(emailInput, 'user@example.com');
  await userEvent.type(passwordInput, 'password123');
  await userEvent.click(submitBtn);

  expect(handleSubmit).toHaveBeenCalledWith({
    email: 'user@example.com',
    password: 'password123'
  });
});
```

7.5 Testing State Changes

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Toggle } from './Toggle';

test('toggle changes visibility', async () => {
  render(<Toggle />);

  // Initially hidden
  expect(screen.queryByText('Secret content')).not.toBeInTheDocument();

  // Click to show
  const button = screen.getByRole('button');
  await userEvent.click(button);

  // Now visible
  expect(screen.getByText('Secret content')).toBeInTheDocument();

  // Click to hide
  await userEvent.click(button);

  // Hidden again
  expect(screen.queryByText('Secret content')).not.toBeInTheDocument();
});
```

Chapter 8

Testing Connected Components

8.1 Testing with Props

```
import { render, screen } from '@testing-library/react';
import { UserProfile } from './UserProfile';

test('displays user profile correctly', () => {
  const user = {
    id: 1,
    name: 'Alice',
    email: 'alice@example.com',
    role: 'admin'
  };

  render(<UserProfile user={user} />);

  expect(screen.getByText('Alice')).toBeInTheDocument();
  expect(screen.getByText('alice@example.com')).toBeInTheDocument();
  expect(screen.getByText('Admin')).toBeInTheDocument();
});
```

8.2 Testing with Context

```
import { render, screen } from '@testing-library/react';
import { ThemeProvider, useTheme } from './ThemeContext';

function ThemedComponent() {
  const { theme } = useTheme();
  return <div>Current theme: {theme}</div>;
}

test('component uses context', () => {
  render(
    <ThemeProvider initialTheme="dark">
      <ThemedComponent />
    </ThemeProvider>
  );
});
```

```
    expect(screen.getByText('Current theme: dark')).toBeInTheDocument();
  });
```

8.3 Testing with Redux or State Manager

```
import { render, screen } from '@testing-library/react';
import { Provider } from 'react-redux';
import { createStore } from 'redux';
import { Counter } from './Counter';

test('counter displays Redux state', () => {
  const initialState = { count: 5 };
  const reducer = (state = initialState) => state;
  const store = createStore(reducer);

  render(
    <Provider store={store}>
      <Counter />
    </Provider>
  );

  expect(screen.getByText('Count: 5')).toBeInTheDocument();
});
```

8.4 Testing with React Router

```
import { render, screen } from '@testing-library/react';
import { BrowserRouter } from 'react-router-dom';
import { Navigation } from './Navigation';

test('navigation links are rendered', () => {
  render(
    <BrowserRouter>
      <Navigation />
    </BrowserRouter>
  );

  expect(screen.getByText('Home')).toBeInTheDocument();
  expect(screen.getByText('About')).toBeInTheDocument();
  expect(screen.getByText('Contact')).toBeInTheDocument();
});
```

Chapter 9

Testing Asynchronous Code

9.1 Testing Promises

```
import { render, screen, waitFor } from '@testing-library/react';
import { UserList } from './UserList';

test('loads and displays users', async () => {
  render(<UserList />);

  // Wait for async operation to complete
  await waitFor(() => {
    expect(screen.getByText('Alice')).toBeInTheDocument();
    expect(screen.getByText('Bob')).toBeInTheDocument();
  });
});
```

9.2 Testing API Calls

```
import { render, screen, waitFor } from '@testing-library/react';
import { DataFetcher } from './DataFetcher';

// Mock fetch
global.fetch = jest.fn(() =>
  Promise.resolve({
    json: () => Promise.resolve({ users: [
      { id: 1, name: 'Alice' }
    ]})
  })
);

test('fetches and displays data', async () => {
  render(<DataFetcher />);

  // Wait for data to load
  await waitFor(() => {
    expect(screen.getByText('Alice')).toBeInTheDocument();
  });
});
```

```
// Check if fetch was called
expect(fetch).toHaveBeenCalledWith('/api/users');
});
```

9.3 Testing Loading States

```
import { render, screen, waitFor } from '@testing-library/react';
import { DataFetcher } from './DataFetcher';

test('shows loading state while fetching', async () => {
  render(<DataFetcher />);

  // Initially shows loading
  expect(screen.getByText('Loading...')).toBeInTheDocument();

  // Wait for data to appear
  await waitFor(() => {
    expect(screen.queryByText('Loading...')).not.toBeInTheDocument();
    expect(screen.getByText('Data loaded')).toBeInTheDocument();
  });
});
```

9.4 Testing Error States

```
import { render, screen, waitFor } from '@testing-library/react';
import { DataFetcher } from './DataFetcher';

global.fetch = jest.fn(() =>
  Promise.reject(new Error('Network error'))
);

test('displays error when fetch fails', async () => {
  render(<DataFetcher />);

  await waitFor(() => {
    expect(screen.getByText(/error/i)).toBeInTheDocument();
  });
});
```

9.5 Testing useEffect

```
import { render, screen, waitFor } from '@testing-library/react';
import { Timer } from './Timer';

test('timer updates after mount', async () => {
  render(<Timer />);
```



```
expect(screen.getByText('Time: 0')).toBeInTheDocument();

await waitFor(() => {
  expect(screen.getByText('Time: 1')).toBeInTheDocument();
});
```

Chapter 10

Working With Mocks

10.1 Understanding Mocks

Mocks replace real functions or modules with fake versions for testing.

10.2 Creating Mock Functions

```
// Creating a mock function
const mockCallback = jest.fn();

// Using the mock
mockCallback('hello');
mockCallback('world');

// Asserting the mock was called
expect(mockCallback).toHaveBeenCalled();
expect(mockCallback).toHaveBeenCalledTimes(2);
expect(mockCallback).toHaveBeenCalledWith('hello');
expect(mockCallback).toHaveBeenNthCalledWith(2, 'world');
```

10.3 Mock Return Values

```
const getUserById = jest.fn();

// Return specific value
getUserById.mockReturnValue({ id: 1, name: 'Alice' });

// Return different values on successive calls
getUserById
  .mockReturnValueOnce({ id: 1, name: 'Alice' })
  .mockReturnValueOnce({ id: 2, name: 'Bob' });

// Return promised value
const fetchUser = jest.fn();
fetchUser.mockResolvedValue({ id: 1, name: 'Alice' });
```

```
// Return rejected promise
const failFetch = jest.fn();
failFetch.mockRejectedValue(new Error('Network error'));
```

10.4 Mocking Modules

```
// math.js
export function add(a, b) {
  return a + b;
}

// math.test.js
jest.mock('./math');
import * as math from './math';

test('mocking a module', () => {
  math.add.mockReturnValue(10);
  expect(math.add(2, 3)).toBe(10);
});
```

10.5 Mocking API Calls

```
import { render, screen, waitFor } from '@testing-library/react';
import { UserFetcher } from './UserFetcher';

// Mock fetch globally
jest.mock('node-fetch');
import fetch from 'node-fetch';

test('fetches user data', async () => {
  fetch.mockResolvedValueOnce({
    json: async () => ({ id: 1, name: 'Alice' })
  });

  render(<UserFetcher userId={1} />);

  await waitFor(() => {
    expect(screen.getByText('Alice')).toBeInTheDocument();
  });

  expect(fetch).toHaveBeenCalledWith('/api/users/1');
});
```

10.6 Mocking Child Components

```
// ChildComponent.js
export function Child({ data }) {
```

```
    return <div>{data.name}</div>;
  }

// Parent.js
import { Child } from './Child';
export function Parent() {
  return <Child data={{ name: 'Test' }} />;
}

// Parent.test.js
jest.mock('./Child');
import { Child } from './Child';
import { Parent } from './Parent';

test('parent renders child', () => {
  Child.mockReturnValue(<div>Mocked Child</div>);
  render(<Parent />);
  expect(screen.getByText('Mocked Child')).toBeInTheDocument();
});
```

10.7 Mocking Window Objects

```
test('mocking window.localStorage', () => {
  const setItemSpy = jest.spyOn(Storage.prototype, 'setItem');
  const getItemSpy = jest.spyOn(Storage.prototype, 'getItem');

  localStorage.setItem('key', 'value');

  expect(setItemSpy).toHaveBeenCalledWith('key', 'value');

  setItemSpy.mockRestore();
  getItemSpy.mockRestore();
});

test('mocking window.alert', () => {
  const alertSpy = jest.spyOn(window, 'alert').mockImplementation();

  window.alert('Test');

  expect(alertSpy).toHaveBeenCalledWith('Test');

  alertSpy.mockRestore();
});
```

10.8 Mock Assertions

```
const mock = jest.fn();

mock('arg1', 'arg2');
```

```
// Various assertions
expect(mock).toHaveBeenCalled(); // Called at all?
expect(mock).toHaveBeenCalledTimes(1); // Called exactly once?
expect(mock).toHaveBeenCalledWith('arg1', 'arg2'); // With these args?
expect(mock).toHaveBeenLastCalledWith('arg1'); // Last call args?
expect(mock).toHaveReturnedWith(value); // Returned this?
expect(mock).toHaveReturnedTimes(1); // Returned exactly once?

// Get call details
console.log(mock.mock.calls); // All calls
console.log(mock.mock.results); // All return values
```

Chapter 11

Common Jest Matchers

11.1 Basic Matchers

```
// Equality
expect(value).toBe(5);
expect(value).toEqual({ name: 'Alice' });

// Boolean
expect(value).toBeTruthy();
expect(value).toBeFalsy();

// Null/Undefined
expect(value).toBeNull();
expect(value).toBeUndefined();
expect(value).toBeDefined();

// Truthiness variations
expect(value).toStrictEqual(otherValue);
```

11.2 Number Matchers

```
expect(4).toBeGreaterThan(3);
expect(4).toBeGreaterThanOrEqual(4);
expect(3).toBeLessThan(4);
expect(3).toBeLessThanOrEqual(3);
expect(0.1 + 0.2).toBeCloseTo(0.3);
```

11.3 String Matchers

```
expect('hello world').toContain('world');
expect('hello world').toMatch(/hello/);
expect('hello world').toMatch('hello');
```

11.4 Array Matchers

```
expect([1, 2, 3]).toContain(2);  
expect([1, 2, 3]).toEqual([1, 2, 3]);  
expect([1, 2, 3]).toHaveLength(3);
```

11.5 DOM Matchers

```
// From @testing-library/jest-dom  
expect(element).toBeInTheDocument();  
expect(element).toBeVisible();  
expect(element).toBeDisabled();  
expect(element).toHaveTextContent('Hello');  
expect(element).toHaveAttribute('href', '/');  
expect(element).toHaveClass('active');
```

Chapter 12

Best Practices & Summary

12.1 Testing Best Practices

- **Test Behavior, Not Implementation:** Focus on what users see, not how code works
- **Use Descriptive Test Names:** `test('displays error when email is invalid')`
- **Keep Tests Simple:** One assertion per test when possible
- **Avoid Implementation Details:** Don't test internal state directly
- **Clean Up After Tests:** Use `afterEach()` to clean up
- **Mock External Dependencies:** Don't test third-party libraries
- **Use Descriptive Query Methods:** Prefer `getByRole` over `getById`
- **Test Edge Cases:** Empty inputs, large numbers, null values
- **Don't Test Third-Party Code:** React, libraries are already tested
- **Aim for 80% Coverage:** Don't obsess over 100%

12.2 Common Mistakes

- **Testing Implementation Details:** Changes break tests unnecessarily
- **Forgetting to Wait for Async:** Use `waitFor()` for async operations
- **Not Cleaning Up Mocks:** Call `mockClear()` between tests
- **Testing Too Much:** Write focused tests, not integration tests disguised as unit tests
- **Not Testing Error Cases:** What happens when things go wrong?
- **Using `setTimeout` in Tests:** Use `jest.useFakeTimers()` instead
- **Testing CSS and Styling:** Leave that to visual testing tools
- **Not Organizing Tests:** Use `describe()` to group related tests

12.3 Testing Checklist

Before considering your tests complete:

- Critical user flows tested
- Error cases handled
- Edge cases covered
- Async operations await properly
- Mocks are used for external dependencies
- Tests are independent (no cross-dependencies)
- Tests run fast (under a few seconds)
- Test names are descriptive
- No implementation details in tests
- Tests follow AAA pattern (Arrange, Act, Assert)

12.4 Test Coverage Goals

- **Line Coverage:** 80% - Most code paths executed
- **Branch Coverage:** 75% - Most if/else branches tested
- **Function Coverage:** 85% - Most functions called
- **Statement Coverage:** 80% - Most statements executed

These are guidelines, not strict rules.

12.5 Quick Reference

Common Testing Commands:

```
npm test           # Run all tests
npm test -- --watch # Watch mode
npm test -- --coverage # Show coverage
npm test FileName  # Run specific file
npm test -- --updateSnapshot # Update snapshots
```

Chapter 13

Conclusion

Testing React applications ensures your code works correctly and catches bugs before users encounter them. By following these practices:

1. Write tests that matter (behavior-focused)
2. Keep tests simple and focused
3. Use the right tools (Jest, React Testing Library)
4. Mock external dependencies
5. Test asynchronous code properly
6. Maintain your tests as your code evolves

Your React applications will be more reliable, easier to refactor, and professional-grade.